

Programming Fundamentals (COSC2531)

Final Coding Project

Assessment Type	Individual assessment (no group work). Submit online via Canvas/Assignments/Final Coding Project. Marks are awarded per rubric (please see the rubric on Canvas). Clarifications/updates may be made via announcements. Questions can be raised via the Canvas discussion forum.
Due Date	End of Week 14 (exact time is shown in Canvas/Assignments/Final Coding Project) Deadline will not be advanced nor extended. Please check Canvas/Assignments/Final Coding Project for the most up to date information regarding the assignment. As this is a major assignment, a university standard late penalty of 10% (i.e., 3 marks) per each day applies for up to 5 days late, unless special consideration has been granted.
Weighting	30 marks out of 100

1. Overview

The main objective of this final project is to assess your capability of program design and implementation for solving a non-trivial problem. You are to solve the problem by designing a number of classes, methods, code snippets and associating them towards a common goal. If you have questions, please ask via the relevant Canvas discussion forums in a general manner; for example, you should replicate your problem in a different context in isolation before posting, and you must not post your code on the Canvas discussion forum.

2. Assessment Criteria

This assignment will determine your ability to:

- i. Follow coding, convention, and behavioural requirements provided in this document and in the course lessons;
- ii. Independently solve a problem by using programming concepts taught in this course;
- iii. Design an OO solution independently and write/debug in Python code;
- iv. Document code;
- v. Provide references where due;
- vi. Meet deadlines;
- vii. Seek clarification from your "supervisor" (instructor) when needed via the Canvas discussion forums; and
- viii. Create a program by recalling concepts taught in class, understand and apply concepts relevant to solution, analyse components of the problem, evaluate different approaches.

3. Learning Outcomes

This assignment is relevant to the following Learning Outcomes:

1. Analyse simple computing problems.
2. Devise suitable algorithmic solutions and code these algorithmic solutions in a computer programming language (i.e., Python).
3. Develop maintainable and reusable solutions using object-oriented paradigm.

4. Assessment Details

Please ensure that you have read Sections 1-3 of this document before going further.

Problem Overview: In this final coding challenge, you are asked to develop a Python program with the Object-Oriented Programming paradigm, named **my_rental.py**, that can read data from files and perform some operations. You are required to implement the program following the below requirements. Note that we will give you some files for you to run with your developed program, BUT you should change the data in these files to test your program. **During the marking, we will use different data/files to test the behavior of your program.**

Requirements: Your code must meet the following **functionalities**, **code**, and **documentation** requirements. Your submission will be graded based on the **rubric** published on Canvas. Please ensure you read all the requirements and the rubric carefully before working on your assignment.

A - Functionalities Requirements:

There are **4 levels**, please ensure you only attempt one level after completing the previous level.

----- **PASS LEVEL (15 marks)** -----

Your project is to implement the required functionalities in the Object-Oriented (OO) style with at least three classes: **RentalLog**, **Bike**, and **Rider**. You need to design appropriate static/instance variables, constructors, and static/instance methods in these classes. The class related info should be encapsulated inside the corresponding class.

At this level, your program can read data from the *rental log file* specified in the command line. The log file records timestamped rental events, where each line represents a BOOKED or RETURNED action for a rider-bike pair. Your program should create a list of *Bike* objects, a list of *Rider* objects, and variables (you can think carefully about which data types to use) to store the rental times for each pair of rider and bike. You should design the classes properly so that these actions can be encapsulated within the appropriate classes. Note that, at this level, we only know the IDs of the bikes and riders. These IDs are all unique.

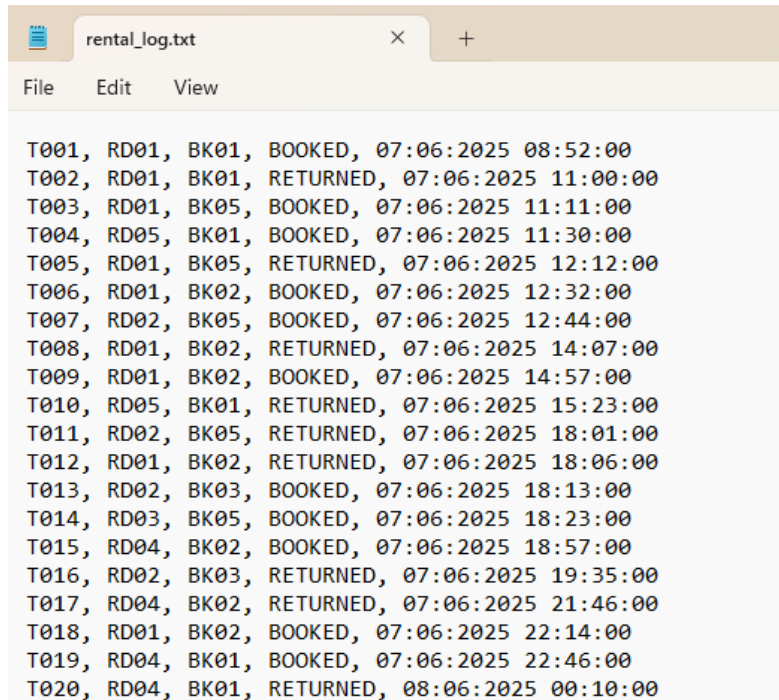
In the main class, your program should create a *RentalLog* object, call its method *load_log(rentallog_filename)* to load all the data from the rental log file, and then call the *show_log()* method to display the rental duration for each combination of bike and rider in the required format (as specified in the following).

Below is an example of the rental log file that stores all the rental transactions (see the next page). Each line in this rental log file has the following format: *transaction_id*, *rider_id*, *bike_id*, *status*, *timestamp*, where:

- *transaction_id* is a unique transaction ID (e.g., T001)

- *rider_id* is the rider ID (e.g., RD01)
- *bike_id* is the bike ID (e.g., BK01)
- *status* is either BOOKED or RETURNED
- *timestamp* is in the format dd:mm:yyyy hh:mm:ss

An example of the rental log file is as follows. You can assume there are no duplicate or redundant rows. You can also assume the format of the data in the file is always correct.



```
T001, RD01, BK01, BOOKED, 07:06:2025 08:52:00
T002, RD01, BK01, RETURNED, 07:06:2025 11:00:00
T003, RD01, BK05, BOOKED, 07:06:2025 11:11:00
T004, RD05, BK01, BOOKED, 07:06:2025 11:30:00
T005, RD01, BK05, RETURNED, 07:06:2025 12:12:00
T006, RD01, BK02, BOOKED, 07:06:2025 12:32:00
T007, RD02, BK05, BOOKED, 07:06:2025 12:44:00
T008, RD01, BK02, RETURNED, 07:06:2025 14:07:00
T009, RD01, BK02, BOOKED, 07:06:2025 14:57:00
T010, RD05, BK01, RETURNED, 07:06:2025 15:23:00
T011, RD02, BK05, RETURNED, 07:06:2025 18:01:00
T012, RD01, BK02, RETURNED, 07:06:2025 18:06:00
T013, RD02, BK03, BOOKED, 07:06:2025 18:13:00
T014, RD03, BK05, BOOKED, 07:06:2025 18:23:00
T015, RD04, BK02, BOOKED, 07:06:2025 18:57:00
T016, RD02, BK03, RETURNED, 07:06:2025 19:35:00
T017, RD04, BK02, RETURNED, 07:06:2025 21:46:00
T018, RD01, BK02, BOOKED, 07:06:2025 22:14:00
T019, RD04, BK01, BOOKED, 07:06:2025 22:46:00
T020, RD04, BK01, RETURNED, 08:06:2025 00:10:00
```

Your program should print a message indicating how to run the program if no rental log file is passed in as a command line argument. Otherwise, it can display a table showing the duration for each rental of a rider and a bike, and two sentences showing the total number of bikes and riders, and the average rental duration.

In the table, if a bike is not rented by a biker, then the data field at that location has an 'xx' symbol. On the other hand, if the bike has been booked by a rider but not yet returned, the data field at that location is shown as a double dash (--). Note that, a bike can be booked by a rider multiple times, so in that particular data field, your program can separate the different rental durations by commas. Also, the data in the columns in the table are aligned, in particular, the Rider IDs are aligned to the left whilst the rental durations are aligned to the right. The rental durations are displayed in full numbers (i.e. no digit after the decimal point).

The printed messages corresponding to two scenarios need to be exactly as below:

1. This is when no rental log file is passed in as a command line argument.

```
>python my_rental.py
[Usage:] python my_rental.py <rental file>
```

2. This is when a rental log file is passed in as command line arguments. Note that users can specify a different file name, not necessarily the name *rental_log.txt*.

```
>python my_rental.py rental_log.txt

RENTAL INFORMATION
+-----+-----+-----+-----+
| Rider ID | BK01 | BK02 | BK03 | BK05 |
+-----+-----+-----+-----+
| RD01 | 128 | 95,189,-- | xx | 61 |
| RD02 | xx | xx | 82 | 317 |
| RD03 | xx | xx | xx | -- |
| RD04 | 84 | 169 | xx | xx |
| RD05 | 233 | xx | xx | xx |
+-----+-----+-----+-----+

Total number of bikes: 4
Total number of riders: 5
Average rental duration: 150.89 minutes
```

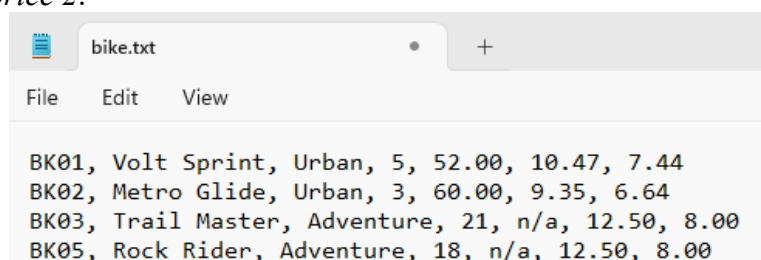
----- CREDIT LEVEL (3 marks, you must only attempt this level after completing the PASS level) -----

At this level, your program can support more information about bikes. Now, apart from the ID, each bike will have a model name, type, gear count, battery range, and the two rental prices per hour. All information (except the type): the IDs, model names, gear counts, battery ranges, and the two rental prices can be modified. There are two types of bikes: Urban and Adventure. All Urban bikes must have the battery ranges larger than 10kms, and each Urban bike will have a different value of battery range. On the other hand, the Adventure bikes will have the battery ranges as *n/a*. The two rental prices should be positive number, and the first rental price should always be larger than the second rental price. The first rental price is applied when the rental duration is less than or equal to 5 hours, the second rental price is applied when the rental duration is larger than 5 hours. All the Urban bikes have different rental prices whilst all the Adventure bikes have fixed values of rental prices as 12.5\$/hour and 8.0\$/hour.

When reading the bike file, if these requirements are violated for any bike, then the program should terminate, and it should also display the message indicating the issue and the corresponding bike. You should define appropriate variables, getters, and setters for bikes.

A bike should have a method to compute some useful statistics, e.g., the number of finished rentals (booked and already returned), the number of ongoing rentals (booked but not yet returned), the total rental duration, etc. (it is your choice to define the necessary statistics to compute). You can define extra methods if necessary.

You can assume there are no duplicate or redundant bikes. You can assume all the bikes available in this file are also available in the rental log file (in the PASS level), and vice versa. An example of the bike file is as follows. The format of each line is *bike ID, model, type, gear count, battery range, rental price 1, and rental price 2*.



```
bike.txt
File Edit View

BK01, Volt Sprint, Urban, 5, 52.00, 10.47, 7.44
BK02, Metro Glide, Urban, 3, 60.00, 9.35, 6.64
BK03, Trail Master, Adventure, 21, n/a, 12.50, 8.00
BK05, Rock Rider, Adventure, 18, n/a, 12.50, 8.00
```

At this level, your program can now also print the bike information table on screen and save the bike information into a file name *rental_records.txt*. For example, given the above rental log file (in the

PASS level) and the bike file (in this level), the bike information table should look like below. Note the bike table within the *rental_records.txt* file should also look the same.

BIKE INFORMATION								
Bike ID	Model	Type	Gear	BatteryRange	Prices	Nfinish	Ngoing	TotalRental
BK01	Volt Sprint	Urban	5	52.00	10.47/7.44	3	0	445.00
BK02	Metro Glide	Urban	3	60.00	9.35/6.64	3	1	453.00
BK03	Trail Master	Adventure	21	n/a	12.50/8.00	1	0	82.00
BK05	Rock Rider	Adventure	18	n/a	12.50/8.00	2	1	378.00

Most valuable bike(s) (by total rental duration): Metro Glide
 Most popular bike(s) (by number of rentals): Metro Glide

In this level, users can pass two file names via the command line arguments: the rental log file and the bike file. The program will then display **two tables**: the rental information table from the PASS level, and the bike information table from this level. Note users can specify different file names, not necessarily the names *rental_log.txt*, and *bikes.txt*. The first file is always the rental log file, and the second file is the bike file.

In the Bike information table, the Bike IDs column shows the IDs of the bikes, the Model column shows the bike models, the Type column shows the types of the bikes, the Gear column shows the number of gears, the BatteryRange column shows the battery ranges of the bikes, the Prices column shows the two rental prices with the format *price_1/price_2*, the Nfinish column shows the number of rentals that already finished renting the bikes (booked and returned), the Ngoing column shows the number of rentals that already booked but haven't returned the bikes, and the TotalRental column shows the total rental duration (in minutes) of the bikes. For all the float numbers in the table, they will need to be displayed with 2 digits after the decimal point.

Apart from this table, your program should also display two sentences. The first sentence indicates the most valuable bike, i.e. the bike that has the highest rental duration. If there are multiple bikes with the same highest rental duration, you can choose to either display one or all bikes. The second sentence indicates the most popular bike, which is the bike with the greatest number of booked rentals. Similarly, if there are multiple bikes with the same number of rentals, you can choose to display one or all bikes.

Finally, it's worth emphasizing that with the program developed in this level, the behavior of command-line arguments is still kept as in the PASS level. Specifically, when the user does not pass any command-line argument, the program will display the usage message, and when the user passes in a rental log filename, the program will display the rental information table. Based on this level, when the user passes in a rental log filename and a bike filename, the program will display the rental information and bike tables.

----- DI LEVEL (4 marks, you must only attempt this level after completing the CREDIT level) -----

At this level, your program can support two types of riders: *Casual* and *Pro*. The type of a rider cannot be changed. Casual riders can rent one Urban and one Adventure bike at a time, and their total daily riding time must not exceed 10 hours. Pro riders can rent one Urban and two Adventure bikes at a time and their total daily riding time must not exceed 18 hours.

A rider should have a method to compute some useful statistics for the rider, e.g., the number of Urban bikes they are booking, the number of Adventure bikes they are booking, the total daily riding time, etc. (it is your choice to define the necessary statistics to compute). It should also have a method to check if a rider satisfies the aforementioned requirement regarding the number of maximum Urban

and Adventure bikes one can rent at one time, and the total maximum daily riding time. You can define extra methods for the riders if necessary.

At this level, the rider file will have more information. The file includes the rider ID, full name, phone number, join date, type. All these information (except the type) can be modified. You can assume there are no duplicate or redundant rows. You can assume all riders in the rental log file (in the PASS level) appeared in this file and vice versa. An example of the rider file is as follows.

```
rider.txt
File Edit View

RD01, Alice Smith, 0412345678, 12/03/2023 09:15:00, Casual
RD02, Benjamin Clark, 0421876543, 25/06/2022 14:30:00, Pro
RD03, Chloe Johnson, 0433567890, 09/11/2021 08:00:00, Casual
RD04, David Miller, 0409123456, 01/01/2024 12:00:00, Pro
RD05, Emma Thompson, 0450765432, 30/09/2023 17:45:00, Casual
```

At this level, users can pass three file names via the command line arguments: the rental log file, the bike file, and the rider file. The program will then display **three tables**: the rental information table from the PASS level, the bike table from the CREDIT level, and the rider table from the DI level. All these tables will also be stored in the text file *rental_records.txt* (from the CREDIT level).

Given the above rental log file (in the PASS level), the bike file (in the CREDIT level), and the rider file (in this level), the rental and bike information tables look the same as in the previous levels whilst the rider information table should look like below. Note the content within the *rental_records.txt* file should also look the same. Also, in the command line, users can specify different file names, not necessarily the names *rental_log.txt*, *bikes.txt*, *riders.txt*. The first file is always the rental log file, the second file is the bike file, and the last file is the rider file.

RIDER INFORMATION									
ID	Name	Type	Phone	JDate	Nfinish	Nongoing	Duration	Cost	
RD01	Alice Smith	Casual	0412345678	12-Mar-2023 09:15:00	3/1	1/0	473.00	79.30	
RD02	Benjamin Clark	Pro	0421876543	25-Jun-2022 14:30:00	0/2	0/0	399.00	59.35	
RD03	ChLoe Johnson	Casual	0433567890	09-Nov-2021 08:00:00	0/0	0/1	0.00	0.00	
RD04	David Miller	Pro	0409123456	01-Jan-2024 12:00:00	2/0	0/0	253.00	40.99	
RD05	Emma Thompson	Casual	0450765432	30-Sep-2023 17:45:00	1/0	0/0	233.00	40.66	
Most active rider(s): Alice Smith with total duration 473.00 mins									
Most valuable rider(s): Alice Smith with total cost \$79.30									

In the rider table, the Rider ID column shows the riders' IDs, Name column shows the riders' names, the Type column shows the riders' types, the Phone column shows the phone numbers, the JDate column shows the joined dates (including the time), the Nfinish column shows the number of bikes finished renting (booked and returned) – with the format of *number of Urban bikes / number of Adventure bikes*, the Nongoing column shows the number of bikes booked but not yet returned – with the format of *number of Urban bikes / number of Adventure bikes*, the Duration column shows the total renting time so far of the rider (not including the duration of bikes that haven't returned), and the Cost column shows the total cost so far of the rider (not including the cost of bikes that haven't returned). All the floating-point numbers in the table need to be displayed with 2 digits after the decimal point. Besides, in the table, if a rider doesn't satisfy the requirement of the maximum number of Urban and Adventure bikes, an exclamation mark (!) will be added to the Nfinish and Nongoing columns. If a rider doesn't satisfy the requirement of the maximum daily riding time for any day, two exclamation marks (!!)

will be added to the Duration column. Apart from the rider information table, two additional sentences are required. The first sentence indicates the most active rider (the rider with the highest rental duration) along with the total rental duration. If there are multiple riders with the highest rental duration, you can choose to display one or

all these riders. The second sentence indicates the most valuable rider (the rider with the highest cost) along with their cost. Similarly, if there are multiple riders with the highest cost, you can choose to display one or all these riders.

Finally, it's worth emphasizing that with the program developed in this level, the behavior of command-line arguments is still kept as in the PASS and CREDIT levels. Specifically, when the user does not pass any command-line argument, the program will display the usage message. When the user passes in a rental log filename, the program will display the rental information table. When the user passes in a rental log filename and a bike filename, the program will display the rental and bike information tables. Based on this level, when the user passes in a rental log filename, a bike filename, and a rider filename, the program will display the rental, bike, and rider information tables.

----- HD LEVEL (5 marks, you must only attempt this level after completing the DI level) -----

At this level, your program can handle some variations in the files using built-in/custom exceptions:

1. When any of the input files cannot be opened (due to missing files, incorrect paths, or permission errors), then your program should print a message indicating the names of the files that have the issue and then quit gracefully. You can assume users always type the file names in the right order in the command line, e.g., the rental log file first, the bike file second, and the rider file third.
2. The program will exit and indicate the corresponding error when one of these errors occurs:
 - a. One of the files passed in via command-line arguments has a wrong number of fields per line;
 - b. In the rental log file, the transaction ID doesn't start with the letter "T" or doesn't have all the remaining characters being numbers, or the timestamp is not in the right date time format as in the spec;
 - c. In the rental log and the bike files, the bike ID doesn't start with "BK" or doesn't have all the remaining characters being numbers, or the prices (in the bike file) are not positive numbers;
 - d. In the rental log and the rider file, the rider ID doesn't start with "RD" or doesn't have all the remaining characters being numbers, or the join date (in the rider file) is not in the right date time format as in the spec.

The program will have some additional requirements (some might be challenging):

- i. In this level, the program will be able to display another section (after the Rider Information table) indicating the number of rentals per 2 hours (defined based on the booking time). Note this is accumulated through different days. An example is as follows.

```

2-HOURLY BOOKING HISTOGRAM
00:00-01:59 |
02:00-03:59 |
04:00-05:59 |
06:00-07:59 |
08:00-09:59 | *
10:00-11:59 | **
12:00-13:59 | **
14:00-15:59 | *
16:00-17:59 |
18:00-19:59 | *****
20:00-21:59 |
22:00-23:59 | ***
  
```

- ii. In this level, the bike information table is split into two tables: one for Urban and one for Adventure. The Urban table consists of all information for the Urban bikes whilst the Adventure table consists of all information for the Adventure bikes. All the formats and information of all the columns are same as with the previous one table. Each table is sorted (from high to low) based on the TotalRental column. Note that this is also reflected in the *rental_records.txt* file.
- iii. In this level, the rider information table is also split into two tables: one for the Casual riders and one for the Pro riders. The Casual rider table consists of all the information for the Casual riders whilst the Pro rider table consists of all the information for the Pro riders. All the formats and information of all the columns are same as with the previous one table. Each table is sorted (from high to low) based on the Cost column. Note that this is also reflected in the *rental_records.txt* file.
1. The *rental_records.txt* now has the date and time when the file was generated (in the format *dd/mm/yyyy hh:mm:ss*, e.g. *07/06/2025 20:12:00*). The rental record file is accumulated and the newest one is at the top whilst the oldest one is at the bottom of the file.

B - Code Requirements:

You must demonstrate your ability to program in Python by yourself, i.e., you should not use any Python packages that can do most of the coding for you. **The only Python packages allowed in this assignment are *sys*, *os*, *datetime***. If other packages/libraries are used, you will get a heavy penalty.

Your program at all levels should be fully OO, e.g., no variables, methods, or code snippets dangling outside a class. Your main program should simply create an object and run its methods to invoke methods from other classes to perform the required functionalities.

You should test/verify the program with different text files (not just run with our text files) to ensure your program satisfies all the required functionalities.

You are not allowed to use Artificial Intelligence tools (e.g., ChatGPT, Copilot) to help you when developing your code. If this is detected by our plagiarism checker, a plagiarism/misconduct case will be open for you, and you will be reported to the RMIT Misconduct team who will then handle your plagiarism/misconduct case.

Your code needs to be formatted consistently. You must not include any unused/irrelevant code (even inside the comments). What you submitted must be considered the final product.

You should design your classes carefully. You may use a class diagram to assist with the design. **In the DI and HD levels, you are required to provide a detailed class diagram to show your class design**. An example of a class diagram is shown below – Figure 1 (note that this is a class diagram of a different programming assignment). In the diagram, the variables and methods of each class are shown. Note that if your code is at the PASS or CREDIT level, you do not need to submit any diagram; a diagram at these levels would NOT result in any mark.

You could use tools like PowerPoint, Keynotes, or online tools like moqups.com to draw the diagram. **The diagram needs to be submitted in jpg, gif, png, or pdf format**.

Finally, note that in places where this specification may not tell you how exactly you should implement a certain feature, you need to use your judgment to choose and apply the most appropriate concepts from our course materials. You should follow answers given by your "client" (or "supervisor" or the teaching team) under Canvas/Discussions/Discussion on Final Coding Project.

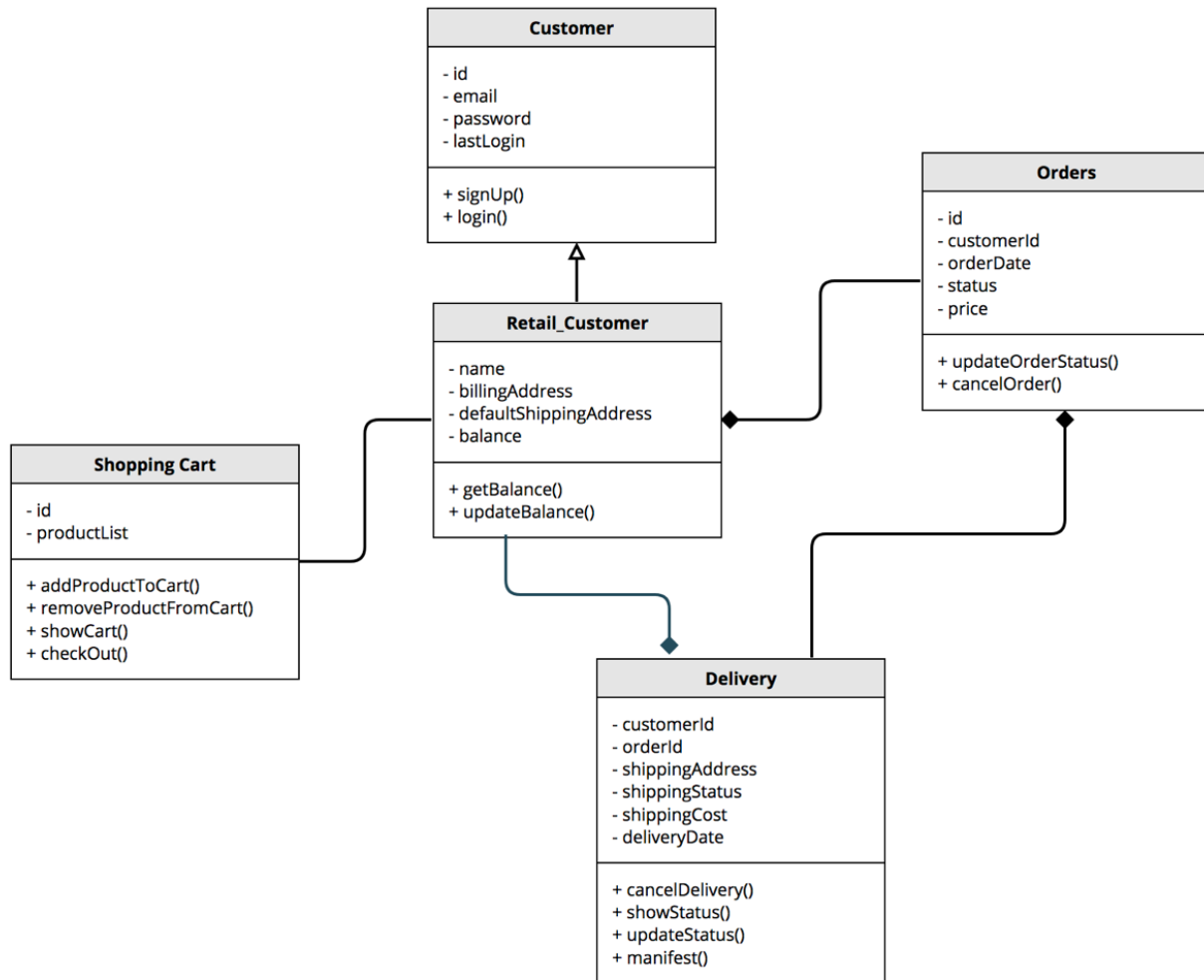


Figure 1. An example of a class diagram

C - Documentation Requirements:

You are required to write comments (documentation) as a part of your code. Writing documentation is a good habit in professional programming. It is particularly useful if the documentation is next to the code segment that it refers to. NOTE that you don't need to write an essay, i.e., you should keep the documentation succinct.

Your comments (documentation) should be in the same Python file. Please DO NOT write a separate file for comments (documentation).

At the beginning of your Python file, your code must contain the following information:

1. **Your name and student ID.**
2. **The highest level you have attempted.** This means you have completed all the requirements of the levels below. Mark will be only given at the lowest level of partial completion. For example, if you completed the PASS level, tried 50% of the CREDIT level, 30% of the DI level, 10% of the HD level, then your submission will be marked at the CREDIT level only (we will ignore the DI and HD levels, so please make sure you fully finish one level before moving to the next one).
3. **Any problems of your code and requirements that you have not met.** For example, scenarios that might cause the program to crash or behave abnormally, the requirements your

program does not satisfy. Note that you do not need to handle or address errors that are not covered in the course.

Besides, the comments in this final coding challenge should serve the following purposes:

- Explain your code in a precise but succinct manner. It should include a brief analysis of your approaches instead of simply translating the Python code to English. For example, you can comment on why you introduce a particular function/method, why you choose to use a while loop instead of other loops, why you choose a particular data type to store the data information. These comments can be placed before the code blocks (e.g., functions/methods, loops, if) and important variable declarations that the comments refer to.
- Document some analysis/reflection as a part of your code. Here, you need to write some paragraphs (could be placed at the end or at the beginning of your code) to explain in detail your design process, e.g., how you came up with the design of the program, how you started writing the code after the design process, the challenges you met during the code development.
- Document the references, i.e., any sources of information (e.g., websites) you used other than the course contents directly under Canvas/Modules, you must give acknowledgement of the sources, explaining how you use the sources in this assignment. More detailed information regarding the references can be found in Section 5.

D - Rubric:

Overall:

Level	Points
PASS level	15
CREDIT level	3
DI level	4
HD level	5
Others (code quality, modularity, comments/analysis)	3

More details of the rubric of this assessment can be found on Canvas ([here](#)). Students are required to look at the rubric to understand how the assessment will be graded.

4. Submission

As mentioned in the Code Requirements, **you must submit only one zip file with the name ProgFunFinal_<Your Student ID>.zip** via Canvas/Assignments/Final Coding Challenge. The zip file contains:

- The main Python code of your program, named **my_rental.py**
- A diagram **in one of the formats: jpg, gif, png, pdf** (if you attempt the DI and HD levels)
- Other Python files written by you to be used by your main application.

It is your responsibility to correctly submit your file. Please verify that your submission is correctly submitted by downloading what you have submitted to see if the file includes the correct contents. The final zip file submitted is the one that will be marked. You do not need to submit the text files (record, book, member) we provide you with – we will mark based on our text files. **NOTE, your code must be able to run under command-line as in the specification. Specifically, if the filenames of the record file, book file, and member file are records.txt, books.txt, and members.txt, respectively, then your program should be able to run under the following command lines:**

1. `python my_rental.py`
2. `python my_rental.py rental_log.txt`

3. `python my_rental.py rental_log.txt bike.txt`
4. `python my_rental.py rental_log.txt bike.txt rider.txt`

If you attempt the PASS level, your program should be able to run the first and second command lines. If you attempt until the CREDIT level, your program should be able to run the first, second, and third command lines. If you attempt until the DI or HD level, your program should be able to run all the four command lines. **NOTE, the filenames specified in the command lines can be different, not necessarily `rental_log.txt`, `bike.txt`, `rider.txt`.** You can assume users always specify the correct order of file names, i.e., the rental log file first, the bike file second, and the rider file third.

Late Submission

All assignments will be marked as if submitted on time. Late submissions of assignments without special consideration or extension will be automatically penalised at a rate of 10% of the total marks available per day (or part of a day) late. For example, if an assignment is worth 30 marks and it is submitted 1 day late, a penalty of 10% or 3 marks will apply. This will be deducted from the assessed mark. Assignments will not be accepted if more than five days late unless special consideration or an extension of time has been approved.

Special Consideration

If you are applying for extensions for your assessment within five working days after the original assessment date or due date has passed, or if you are seeking extension for more than seven days, you will have to apply for Special Consideration, unless there are special instructions on your Equitable Learning Plan.

In most cases you can apply for special consideration online [here](#). For more information on special consideration, visit the university website on special consideration [here](#).

5. Referencing Guidelines

What: This is an individual assignment, and all submitted contents must be your own. If you have used sources of information other than the contents directly under Canvas/Modules, **you must give acknowledgement of the sources, explaining in detail how you use the sources in this assignment, and give references using the [IEEE referencing format](#).**

Where: You can add a code comment near the work (e.g., code block) to be referenced and include the detailed reference in the IEEE style.

How: To generate a valid IEEE style reference, please use the [citethisforme](#) tool if you're unfamiliar with this style.

6. Academic Integrity and Plagiarism (Standard Warning)

Academic integrity is about honest presentation of your academic work. It means acknowledging the work of others whilst developing your own insights, knowledge, and ideas. You should take extreme care that you have:

- Acknowledged words, data, diagrams, models, frameworks and/or ideas of others you have quoted (i.e., directly copied), summarized, paraphrased, discussed, or mentioned in your assessment through the appropriate referencing methods.

- Provided a reference list of the publication details so your readers can locate the source if necessary. This includes material taken from the internet sites.

If you do not acknowledge the sources of your material, you may be accused of plagiarism because you have passed off the work and ideas of another person without appropriate referencing, as if they were your own.

RMIT University treats plagiarism as a very serious offence constituting misconduct. Plagiarism covers a variety of inappropriate behaviors, including:

- Failure to properly document a source
- Copyright material from the internet or databases
- Collusion between students

For further information on our policies and procedures, please refer to the University website ([link](#)).

7. Assessment Declaration:

When you submit work electronically, you agree to the assessment declaration:

<https://www.rmit.edu.au/students/student-essentials/assessment-and-results/how-to-submit-your-assessments>